

Shipay — Questão 3

Estratégia de Testes do Scheduler

Foco: garantir ≥ 1.000 requisições/s e $P99 \leq 30$ ms

1. Objetivo

A questão apresenta um scheduler utilizado por um workflow de renderização e solicita uma estratégia de testes que permita garantir as duas premissas explícitas de performance: suportar pelo menos 1.000 requisições por segundo e manter P99 de latência de até 30 ms nas requisições.

O teste principal deve, portanto, exercitar o endpoint **POST /v1/render/scheduler** sob carga sustentada. Os testes de stress, spike e soak complementam essa validação, aumentando a confiança sobre o comportamento do serviço em condições diferentes do cenário nominal.

2. Fluxo considerado

Com base no código fornecido, o endpoint recebe um `RequestBody`, cria uma `Queue RQ` conectada ao `Redis` e agenda o job por meio de `enqueue_at()`, passando `scheduler_datetime`, `publish_event` e `event_content`.

- Workflow → POST /v1/render/scheduler
- Autenticação e validação do schema
- RQ Queue → Redis
- `enqueue_at(scheduler_datetime, publish_event, event_content)`
- Após o horário agendado: RQ Worker executa o job
- `publish_event(event_content)` → Kafka → Subscriber → continuação do workflow

O código fornecido não mostra a implementação do worker, de `publish_event()` ou do subscriber. Por isso, esses componentes são considerados no fluxo de forma complementar, sem presumir detalhes de implementação.

3. Teste principal — Load Test

Este é o teste que valida diretamente o requisito da questão. Eu utilizaria uma ferramenta de geração de carga HTTP, como Locust, para enviar requisições ao endpoint /v1/render/scheduler.

Procedimento:

- Criar um cenário de teste em Python para chamar o endpoint com payloads representativos do `RequestBody`.
- Executar um período inicial de warm-up. Esse período serve para permitir que a aplicação e a infraestrutura entrem em regime; seus dados não são usados na medição oficial.
- Aplicar uma carga sustentada de 1.000 requisições por segundo durante uma janela de medição definida.
- Coletar throughput, P99 de latência e taxa de erro; P50/P95 podem ser observados como métricas auxiliares para entender a distribuição da latência.
- Monitorar também CPU, memória e indicadores do RQ/Redis para identificar gargalos.
- Repetir o teste em condições controladas para verificar a consistência do resultado.

4. Métricas e critérios

A métrica de P99 deve ser medida sobre a requisição HTTP do scheduler: do envio do request até o recebimento da resposta. O processamento posterior é assíncrono e deve ser acompanhado separadamente.

Métrica	Objetivo	Interpretação
Throughput	≥ 1.000 req/s	Requisito explícito de capacidade
P99	≤ 30 ms	Requisito explícito de latência
Taxa de erro	Observar	Identificar falhas sob carga; a questão não define um limite

P50 / P95	Observar	Diagnóstico da distribuição de latência
-----------	----------	---

5. Testes complementares

5.1 Stress Test

A carga é aumentada progressivamente, por exemplo, 500 → 750 → 1.000 → 1.250 → 1.500 req/s, até identificar o ponto em que latência, erros ou throughput começam a degradar. O objetivo é descobrir a margem e o limite do serviço.

5.2 Spike Test

A carga sofre um aumento abrupto, por exemplo, de 100 para 2.000 req/s, e depois retorna à carga normal. São observados P99, erros, backlog e recuperação do serviço.

5.3 Soak / Endurance Test

Mantém-se uma carga próxima de 1.000 req/s por um período prolongado. O objetivo é detectar degradação progressiva, vazamentos de memória, crescimento de backlog ou aumento gradual do P99.

6. Validação complementar do fluxo assíncrono

Como o enunciado destaca que o workflow é essencial e deve ser resiliente, durante os testes de performance eu acompanharia o processamento assíncrono, sem transformar essa parte no requisito principal.

- Jobs processados por segundo
- Backlog no RQ/Redis
- Tempo entre o vencimento do agendamento e sua execução
- Throughput e lag do Kafka
- Capacidade de processamento do subscriber

A ideia é verificar se o sistema consegue absorver a taxa de entrada sem criar um backlog que cresça indefinidamente.

7. Como eu executaria os testes

O Locust seria utilizado como gerador de carga e também como fonte das métricas HTTP do teste. Eu não trataria o Locust como responsável por monitorar toda a infraestrutura: métricas de CPU, memória, Redis/RQ e Kafka seriam obtidas pelos mecanismos de observabilidade disponíveis no ambiente.

- Preparar um ambiente controlado e próximo das condições esperadas de produção.
- Criar o cenário no Locust e utilizar payloads representativos.
- Executar warm-up.
- Executar a janela oficial de medição.
- Coletar throughput, P99, taxa de erro e métricas de infraestrutura.
- Repetir o cenário para confirmar consistência.
- Executar stress, spike e soak como testes complementares.
- Documentar resultados, gargalos observados e o ponto de capacidade encontrado.

8. Experimento local realizado

Como o repositório do desafio não contém a implementação do scheduler, foi criado um pequeno experimento local apenas para validar, na prática, a geração de carga com Locust e a coleta de métricas HTTP. O experimento utilizou FastAPI, RQ e Redis em uma única máquina.

Foram observados cenários de aproximadamente 10 req/s e 50 req/s, além de uma tentativa de elevar a carga utilizando maior concorrência. O experimento mostrou, na prática, a diferença entre taxa-alvo e throughput efetivamente alcançado, o comportamento dos percentis e a necessidade de monitorar os recursos do próprio gerador de carga.

Limitação: os resultados desse laboratório local não devem ser utilizados para afirmar a capacidade de produção de 1.000 req/s, pois o gerador de carga e os componentes do serviço compartilharam a mesma máquina. Em um teste de capacidade real, o gerador de carga deve ser separado da infraestrutura sob teste e os recursos de ambos devem ser monitorados.

9. Sobre o Locust

O Locust pode ser utilizado como ferramenta de geração de carga HTTP. O cenário pode ser escrito em Python, definindo as requisições HTTP que os usuários virtuais executarão. A ferramenta apresenta estatísticas de requests, falhas, throughput e percentis de latência, incluindo P99.

No experimento local, foi utilizado controle de taxa por usuário para aproximar uma taxa-alvo de requisições por segundo. O número de usuários virtuais é um meio de gerar a carga e não representa, por si só, o requisito de 1.000 RPS.

10. Critério final

A premissa principal será considerada atendida quando, sob carga representativa e sustentada, o serviço conseguir manter throughput de pelo menos 1.000 requisições por segundo e P99 de latência de no máximo 30 ms. A taxa de erro será reportada como indicador adicional de estabilidade, sem inventar um limite que não foi definido no enunciado.